Granite — Complete FYP Preparation Guide

Project	F26-060-Granite · Stream A (research-derived) · Proposal defense stage
Team	Shahoud Shahid (23i-2515) · Muhammad Noor (23i-2520) · Saif Shahzad (23i-2634)
Supervisor	Dr. Adil Majeed · FAST School of Computing, NUCES Islamabad

How to read this document

This is written for someone learning the project from scratch. It assumes no background.

Four labels are used throughout, and they mean exactly what they say:

Label	Meaning
CONFIRMED	Already built and run. There is a file in the repository you can open.
PLANNED	Committed work, not built yet.
ASSUMPTION	We believe this is true but have not proven it.
RISK	A known way this could go wrong.

Nothing in this document is dressed up. Where the evidence is thin, it says so. That is deliberate — a panel finds weak claims faster than you can defend them, and the strongest position in a defense is having said the weakness first.

1. The Project Story

1.1 The problem, in plain words

Software is increasingly built from **multi-agent AI systems** — several AI programs working together, each doing one part of a job and passing results to the next.

> **Multi-agent system:** several AI agents that split a task between them and pass information along a chain, like a small team.

A simple example, and this is a real captured run from our own work:

```
User asks: "What time does Riverside Hardware close today,
            and how far is it from downtown?"

  Planner      →  breaks the request into 2 sub-steps
  StoreChecker →  calls a store-lookup tool, gets the hours
  Writer       →  writes the final sentence for the user
```

The team answered: *"Riverside Hardware's closing time is 6:15 PM, which is 33 minutes before the end of the business day, and it is 3.2 miles away from downtown."*

That answer is **wrong**. The real closing time was 9:40 PM. The StoreChecker returned a corrupted value, and the Writer repeated it — and then invented a confident-sounding justification for it ("33 minutes before the end of the business day").

Now the important part. **Nothing crashed.** No error was raised. No exception, no retry, no timeout, no warning. The system looked like it worked perfectly.

Two questions follow, and today nobody can answer either one reliably:

1. **Which agent caused this?** With three agents in a chain, the wrong answer surfaces at the Writer — but the Writer did its job correctly given what it was handed. The real fault is one step upstream.
2. **If we change something to fix it, did the fix actually work?** Or did we just move the problem somewhere else?

1.2 Why it matters

Multi-agent systems are moving into production right now — coding assistants, customer-support agents, research tools. When one produces a confident wrong answer with no error signal:

- **Nobody notices.** There is no alert, because nothing failed in a way software knows how to detect.
- **A person or another system acts on it.** In a refund pipeline, that is money moving on a wrong number.
- **Debugging is manual and slow.** An engineer reads the whole trace by hand and guesses.

How good is that manual guess? The research field measured it: **14.2% accuracy** at identifying the exact failing step. That is the baseline this whole field is trying to beat.

1.3 What research already exists

This is a **very new field**. Before April 2025, automated failure attribution for multi-agent systems barely existed as a research problem.

> **Failure attribution:** working out which agent, and which step, caused a failure.

Then it moved fast. Seven papers in about 15 months, across four top conferences:

```
Apr 2025   Who&When (ICML)          ← created the problem and the first benchmark
Sep 2025   AgenTracer (ICLR 2026)   ← automated the labelling cost away
2026       SDBL (AAAI 2026)         ← two-stage "narrow, then localise"
Jul 2026   TraceElephant (ACL)      ← the bottleneck is missing trace data
Jul 2026   ErrorProbe (ACL Findings)← strongest published method  ★ our baseline
Jul 2026   AgentAsk (ACL)           ← prevent errors at handoff, independent line
Jul 2026   PROTEA (ACL Demo)        ← Sony's interactive debugging tool
```

CONFIRMED: all seven read end-to-end, not from abstracts. Every limitation quoted in Section 3 comes from that paper's own Limitations section.

1.4 Where the existing work stops

Three things the field has not done. Each is backed by the papers' own admissions, not our opinion.

(a) The problem is nowhere near solved. Best published step-level accuracy is **59.5%** (ErrorProbe). After 15 months and seven papers, the best method in the world still gets the failing step wrong about **4 times in 10**.

(b) Silent failures are invisible by design. ErrorProbe's own Limitations section says it fails on outputs that are technically valid but semantically wrong and raise no anomaly signal. This is not an oversight — it is structural. Every method in this lineage starts by looking for something that went visibly wrong, then reasons backwards from it. If nothing looks wrong, the search never starts.

> **Silent failure:** a confident, well-formed, complete — and wrong — answer, with no error, no retry, no warning. The Riverside Hardware case above is exactly this.

(c) Nobody has tested whether these methods generalise. Every paper evaluates on multi-agent architectures that were used in building its own benchmark. None tests transfer to an architecture it has never seen. **CONFIRMED** after full-text reading of all seven.

1.5 The research gap, in one sentence

> No system — academic or commercial — confirms step-level attribution accuracy **together with** general silent-failure detection, **and** verification that a fix actually works, **and** generalisation to an unseen architecture, evaluated as one pipeline.

The gap is the combination, not any single missing feature. This distinction matters and Section 4 defends it properly.

1.6 Our proposed solution

Granite — an installable SDK that reads the execution trace of a multi-agent run and answers three questions:

```
TRACE IN
   │
   ▼
┌──────────────────────────────────────────┐
│ 1. DID it actually fail?                 │
│    Semantic Error Diagnostic             │
│    Checks the answer against the task's own │
│    constraints. Runs on EVERY trace, not │
│    only ones that look broken.           │
├──────────────────────────────────────────┤
│ 2. WHICH agent and step caused it?       │
│    Backward Causal Tracer                │
│    Walks the dependency graph backwards  │
│    from the symptom to the origin.       │
├──────────────────────────────────────────┤
│ 3. Does a proposed fix actually WORK?    │
│    Repair Agent + Verified Automatic Repair │
│    Re-runs the downstream steps with the │
│    fix. Applies it only if the replay    │
│    passes.                               │
└──────────────────────────────────────────┘
   │
   ▼
DIAGNOSIS + VERIFIED FIX OUT
```

1.7 What is genuinely new — stated carefully

We do not claim a better attribution algorithm. We adopt ErrorProbe's published method as our baseline. Beating 59.5% is not the contribution, and claiming it would be unserious at this stage.

Three things we do claim:

#	Claim	Why it is defensible
1	**Silent-failure detection that is not gated on an anomaly**	The constraint check runs unconditionally and in parallel with the tracer, never behind it. Where no machine-checkable constraint exists it **abstains**, and we report the abstention rate as a metric — so the boundary of the claim is measured, not asserted.
2	**Repair verified before it is applied**	Controlled replay with recovery rate measured over N runs. PROTEA proposes fixes but a human approves every one and no recovery rate is reported.
3	**Cross-architecture generalisation**	CrewAI + LangGraph — used by none of the seven papers. This is the actual test of whether these methods generalise.

Honest framing (and say this before a panel says it to you): none of the individual techniques are new. Rule-based fault injection is AgenTracer's. Structural graph parsing is standard. Constraint checking is ordinary software engineering. The contribution is **the measurement that combination produces** — a number nobody has produced. Either the pipeline holds up on an architecture it was never tuned for, or it does not. Either the constraint check catches silent failures, or it does not.

1.8 What the finished product is

A **deployable SDK plus a diagnosis dashboard**, not a research prototype:

- `pip install`, point it at your CrewAI or LangGraph workflow, no changes to your agents
- It captures traces, diagnoses failures, proposes repairs and verifies them
- A dashboard shows the execution graph, the culprit step, the evidence, and the replay result
- A published cross-architecture benchmark (500–1,000 labelled traces) released for others to use

---

2. Feature Table

2.1 How to read the status column

Status	Meaning
Exists (adopted)	Published technique we reuse. Not claimed as ours.
Improving	Exists, but we extend or re-target it.
New	Genuinely not done in the reviewed work — with the caveat in Section 4.1.
Extension	Only if the core lands first. Not promised.

2.2 Tier 1 — core, each backed by a POC that ran

#	Feature	Problem solved	How it works	Implementation plan	Data / model needed	Novelty	Expected output	CCP
1	**Fault-Injection Harness**	Labelled failure data is expensive — the field paid ~30 hrs/annotator	Deterministically corrupts one value in a running workflow. The injection operator **is** the ground-truth label, so no human annotation	Rule-based operators, zero LLM calls. Extend from 1 fault type to 3	GSM8K, tau-bench retail	Exists (adopted from AgenTracer)	Trace + exact ground truth	3, 8
2	**Cross-Architecture Benchmark**	No paper tests an unseen architecture	Run injected faults through CrewAI **and** LangGraph, both unused by all reviewed papers	500–1,000 traces, 15–20% clean controls, published with a datasheet	GSM8K + tau-bench via CrewAI/LangGraph	**New** (data contribution)	Public labelled benchmark	3, 5, 7
3	**Culprit Agent**	"Which agent broke it?"	Hypothesises the responsible agent from the execution graph	Adopt ErrorProbe's Strategist stage	Who&When (184 cases)	Exists (adopted)	Agent ID + confidence	2, 3
4	**Error Step Localization**	"Which *step*?" — the hard half	Hypothesises the exact failing step	Adopt ErrorProbe's Strategist stage	Who&When	Exists (adopted)	Step index + confidence	2, 3, 9
5	**Verification Agent**	Stops the system trusting its own guess	Checks a hypothesis against real tool-call evidence in the trace before committing	Adopt ErrorProbe's Investigator stage	Who&When	Exists (adopted)	Supported / not supported	2, 3
6	**Repair Agent**	Nothing produced the corrected value	Generates one corrected **atomic field** for the localised failure	LLM-generated, schema-constrained	Local model (Ollama)	Improving	Candidate fix	1, 5
7	**Semantic Error Diagnostic**	**Silent failures — the headline gap**	Recomputes the answer's checkable constraints from the task spec and compares. Runs **unconditionally**, abstains when no constraint exists	Rule-based, **zero LLM calls in the checker**	GSM8K numerics, tau-bench policy rules	**New** (at the identified gap)	Flag / pass / abstain	1, 2, 5, 7, 9
8	**Verified Automatic Repair**	Nobody checks a fix before trusting it	Re-runs downstream under a controlled replay protocol (frozen upstream context, pinned decoding). Applies the fix only if the replay passes	Replay engine + recovery rate over N runs	Own benchmark	**New** (at the identified gap)	Recovery rate + variance	1, 4, 9

2.3 Tier 2 — committed core, but with the weakest evidence

Read this honestly. These two have pre-registered decision gates because their evidence is thin. Do not present them as equal to Tier 1.

#	Feature	Problem solved	Current evidence	Gate (must pass by Mid-1)	Recorded fallback	CCP
9	**Recovery Controller**	Resume from a repaired point instead of restarting the whole workflow	**NO POC AT ALL.** Genuinely untested mechanical work	Checkpoint/resume PoC on LangGraph (chosen because its native checkpointing is lowest-risk)	Re-scope to "resumption via controlled replay" — a capability Tier 1 already delivers	1, 8
10	**Verified Repair Memory**	Reuse fixes that worked before	**ONE TEST, AND IT FAILED.** 0 of 8 cases committed anything to memory; all 24 verifications returned NOT_SUPPORTED; agent accuracy regressed 75%→50%	Re-test on a stronger backbone, same 8 cases. Pass = ≥1 committed memory **and** no accuracy regression	Ships as a memory-on vs memory-off **ablation study** — an evaluation contribution, not a silently dropped feature	3, 4

> **Naming trap a panel may spring:** Granite's *Verified Repair Memory* stores **error→repair patterns**, sits **after** verification, and exists to reuse fixes. ErrorProbe's *Verified Episodic Memory* stores **diagnosis evidence**, sits **inside** attribution, and we reproduce it only as part of the baseline. Different content, different position, different purpose — not a rename.

2.4 Extensions — only after the core works

Not promised. Do not present these as deliverables.

Priority	Extension	Why it is wanted	Status
1	Widen fault taxonomy to 3 types	One validated type is thin	Planned
2	Third architecture	Strengthens generalisation from 2 to 3	Stretch
3	Cyclic/dynamic graph support	**LangGraph natively loops** — without this our own cross-architecture claim is incomplete on a framework we committed to	**POC ran (Spike 6)**
4	Confidence calibration	Publish a reliability diagram	Planned
5	Preventive clarification classifier	Catch errors before they propagate	**POC ran and FAILED (0.25)**
6–8	Propagation map · explainable confidence · compatibility layer	Product polish	Stretch

2.5 The honest summary of novelty

Category	Items
Adopted, not ours	ErrorProbe's 3-stage attribution · AgenTracer's fault injection · structural graph parsing · constraint checking
We improve / re-target	Repair Agent (schema-constrained atomic fields) · cyclic graph handling
Genuinely new (with caveats)	Unconditional silent-failure check with reported abstention · verified repair with measured recovery rate · cross-architecture benchmark on CrewAI + LangGraph
Only proposed	All 8 extensions, Recovery Controller, Verified Repair Memory

3. Literature Review

3.0 A correction that has now been applied

Several project documents used to say **"9+ papers read in full."** That was wrong, and it has been corrected across the repository.

Where the number came from. `Proposal_Defense_Package.md` spelled it out: *"9+ papers read in full (Who&When, AgenTracer, SDBL, TraceElephant, ErrorProbe, AgentAsk, PROTEA, plus GraphTracer/FALAT/DoVer triaged)."*

That is **seven read in full, plus three triaged** at title and abstract level. The error was never the count of papers *touched* — it was attaching "read in full" to all of them.

The accurate claim:

Read end-to-end	**7** — Who&When, AgenTracer, SDBL, TraceElephant, ErrorProbe, AgentAsk, PROTEA
Triaged (title/abstract only)	3 — GraphTracer, FALAT, DoVer
Commercial platforms surveyed	7
Prior FYP titles checked	660, across 12 cohorts

What to say if asked "how many papers?":

> *"Seven read end-to-end. Three more triaged at abstract level and logged as a watch list, because this field publishes every two to three months. Plus seven commercial platforms and a 660-title departmental archive check."*

Being able to name all seven, and to say which three were only triaged, is stronger than a bigger number you cannot break down.

3.1 The seven papers, one by one

Paper 1 — Who&When (ICML 2025, 30 Apr 2025)

Problem	Nobody had defined "which agent caused this failure" as a research problem
Approach	Built the first human-annotated benchmark plus three baseline methods (All-at-once, Step-by-step, Binary search)
Key contribution	Created the field. First real annotated benchmark
Limitations (its own words)	Step accuracy only **14.2%** best-case; states even o1 and DeepSeek-R1 "fail to achieve practical usability"
Does not solve	Anything close to usable accuracy. No repair, no silent failures
Relevance to us	The origin of the field, and the source of our 14.2% manual-baseline figure
What we take	The dataset — **184 annotated cases, MIT licensed, fully downloaded.** The only dataset in the lineage we can rely on completely
How we differ	We target silent failures and verified repair, neither of which it addresses

Paper 2 — AgenTracer (ICLR 2026, 3 Sep 2025)

Problem	Annotating failures by hand costs about 30 hours per annotator
Approach	Programmatic fault injection plus counterfactual replay to generate labels automatically; trained an 8B model
Key contribution	Removed the human annotation cost. Showed a small trained model beats far larger proprietary LLMs here
Limitations (its own)	Step accuracy still modest even trained (**~37-43%**); only tested inside its own 6-framework ecosystem; quality capped by its analyser LLM
Does not solve	Silent failures. Generalisation outside its own ecosystem
Relevance	The methodology template for our benchmark
What we take	**The fault-injection idea: the injection operator IS the ground-truth label.** No manual annotation needed
How we differ	We inject into CrewAI and LangGraph, which it never used

CRITICAL	**Its dataset TracerTraj-2.5K was never actually released.** ErrorProbe's paper described it as Apache 2.0; direct repo inspection found **no LICENSE file at all** and only a 127-row sample. This is our standing lesson: *a citation is not a licence check*

Paper 3 — SDBL (AAAI 2026)

Problem	Searching a whole trace for one failing step is inefficient
Approach	Two stages: delineate the scope, then localise. Inspired by how expert humans debug differently from novices
Key contribution	Turned observed expert debugging behaviour into an algorithm. Introduced "Overstep" and "Loop" heuristics
Limitations (its own)	Best step accuracy still only **~33-40%**
Does not solve	The majority of cases are still misattributed
Relevance	A third independent descendant of Who&When
What we take	Mainly evidence: ErrorProbe beats it on identical splits, which strengthens our baseline choice
How we differ	Same as the rest: no silent failures, no repair

Paper 4 — TraceElephant (ACL 2026 Long)

Problem	Everyone assumed weak algorithms; maybe the real problem was missing data
Approach	Captured full input **and** output traces instead of outputs only
Key contribution	Showed the bottleneck was partly **observability**, not reasoning. Full traces improve accuracy up to 76.5% relative to partial observation
Limitations (its own)	**Proposes no attribution algorithm at all.** It is a data and observability contribution only
Does not solve	Attribution itself
Relevance	Justifies why our SDK captures complete traces rather than sampling
What we take	The design principle: capture everything
How we differ	We build the diagnosis method it deliberately did not

Paper 5 — ErrorProbe (ACL 2026 Findings) — OUR BASELINE

Problem	Prior methods were shallow and had no memory
Approach	Three stages: MAST-taxonomy structural decomposition, then symptom-driven backward tracing, then a Strategist / Investigator / Arbiter verification team. Plus Verified Episodic Memory, which commits a failure pattern only once confirmed by executable evidence
Key contribution	The most mechanistically sophisticated of the seven. Its memory design explicitly prevents hallucinated error-pattern pollution
Limitations (its own words)	**Explicitly fails on SILENT FAILURES** — technically valid but semantically wrong outputs raising no anomaly signal. Also adds real latency; validated on only 3 backbone families
Does not solve	**Silent failures. Repair verification. Unseen architectures.** All three of our claims
Relevance	**Our chosen baseline.** It beats both its ancestors on their own benchmarks
What we take	Its published attribution method, as our algorithmic foundation
How we differ	We add the unconditional constraint check, verified repair, and cross-architecture testing
Why this baseline	Chosen on **performance, not recency** — it is not even the newest of the four ACL papers

Paper 6 — AgentAsk (ACL 2026 Long)

Problem	Fixing errors after the fact is too late
Approach	Edge-level failure taxonomy (Data Gap, Signal Corruption, Referential Drift, Capability Gap) plus an RL-trained clarification module that intervenes **at a handoff, before** an error propagates
Key contribution	Preventive rather than post-hoc. Novel E-GRPO training. Architecture-agnostic

Limitations (its own)	**Cannot fix errors originating inside an agent's own reasoning** — it only sees what is visible AT a handoff
Does not solve	Hallucination or reasoning errors inside a single agent, which is where silent failures usually start
Relevance	Independent of the Who&When lineage entirely: different benchmarks, no citation overlap
What we take	The idea, tested as extension #5 — **and our test failed (0.25, near-random)**
How we differ	We are reactive by design, and Spike 7 is the evidence for that choice

Paper 7 — PROTEA (ACL 2026 Demo, Sony)

Problem	Developers need a practical tool, not a method paper
Approach	Full interactive tool: evaluate, localise the bottleneck, propose a prompt fix, auto-rerun, compare before and after. Formative user study, n=6
Key contribution	Closest thing in the corpus to a real product
Limitations (its own)	**No automated root-cause confidence scoring — a human must approve every fix. Supports only static DAG workflows, explicitly not cyclic**
Does not solve	Automated verification. Looping workflows
Relevance	Evidence that industry wants this, and that human-gated repair is the accepted norm, which is our ethical stance too
What we take	Validation that the product shape is right
How we differ	We measure a recovery rate; it reports none. Its DAG-only limit is exactly why our cyclic handling (Spike 6) matters
Note	Evaluated at much lower rigour than the academic papers: two internal case studies, no F1 / precision / recall against ground truth

3.2 The two clusters — useful if a panel says "this area is crowded"

```
CLUSTER 1 - post-hoc attribution
  Who&When (Apr 2025)
     |--> AgenTracer      (cites Who&When directly)
     |--> SDBL            (adopts its three baselines)
     |--> TraceElephant   (reuses its underlying systems)
             |--> ErrorProbe  (evaluates against BOTH Who&When AND TracerTraj)

CLUSTER 2 - unrelated, no citation overlap
  AgentAsk  - preventive, at message handoffs
  PROTEA    - Sony industry debugging tool
```

Two unrelated clusters, not one crowded field.

3.3 The commercial platforms

Seven checked. None is a research method — they are observability products.

Platform	What it does	Root-cause attribution	Silent failures	Fix verified before applying
LangSmith (core)	Trace capture, evals	Human-driven	No	No
LangSmith Engine (Beta 2026)	Diagnoses against the live codebase, drafts a fix as a pull request	**Partial, real** — but the multi-agent step-level mechanism is not publicly documented	Not confirmed	**Not confirmed**
Langfuse	Agent graphs, nested spans. MIT, self-hostable	Human-driven — its own materials leave the developer "to determine which step was responsible"	No	No
Braintrust	Full span tree; converts failures into regression tests	Human-driven, manual root-cause labelling	No	No
Arize Phoenix	OpenTelemetry-based	Human-driven, LLM-assisted evals	No	No

Platform	What it does	Root-cause attribution	Silent failures	Fix verified before applying
Arize "Alyx" (2026)	"Uncovers root causes, generates review-ready fixes"	**Marketing-stage, undocumented mechanism**	Not confirmed	Not confirmed
Maxim AI	Full lifecycle trace graph	Human-assisted; re-runs simulations to *reproduce* an issue	**Partial** — RAG-groundedness only, not general	Not found
Opik / DeepEval	Logging and evaluations	Unconfirmed	Unconfirmed	Unconfirmed

The honest positioning: automated diagnosis IS emerging commercially (LangSmith Engine, Arize Alyx, both 2026). This **sharpens** our differentiation rather than eliminating it — none publicly confirms counterfactual fix-testing or general silent-failure detection. We position Granite as a **diagnosis layer complementary to these platforms**, not a competing observability product.

3.4 The closest prior FYP

CONFIRMED: all 660 titles across 12 published cohorts, Spring 2019 to Fall 2025, checked structurally rather than by title.

`S24-011-D-HDLM` **— "Hallucination Detection in Language Model Textual Responses" (Spring 2024)**

Same underlying problem	AI output that looks fine and is wrong, with no error signal
Its scope	**One** language model's text output
What it lacks	No agent-level or step-level attribution (only one model to blame). No propagation modelling (nothing to propagate through). **No repair or replay** — detection only
Coverage of Granite	At most **one of four pipeline stages**, and that one scoped narrower

Runners-up, named for completeness: `F25-098-R-Questor` (multi-agent fraud detection, closest on architecture), `F25-112-D-GitCortex` (agentic bug triage, closest on automated diagnosis), `F25-208-D-HalluciLens` (hallucination browser extension).

Say this before a panel asks: *"Yes, a prior FYP covered hallucination detection. It confirms the problem is real and department-recognised. It covers one of our four stages, on a single model's text. A hallucination detector for one model has no notion of which agent, which step, or whether a fix works — because none of those concepts exist without multiple interacting agents."*

3.5 Comparison matrix

Paper / Application	What it does	Limitation	Our difference	Our improvement
Who&When	Defined the problem, first benchmark	14.2% step accuracy	We adopt the dataset, target a different failure class	Silent failures plus repair
AgenTracer	Auto-labelled data via fault injection	~37-43%; own ecosystem only; **dataset unreleased**	We use its method on unseen frameworks	Our benchmark is actually released
SDBL	Two-stage narrow-then-localise	~33-40%	Not our baseline	—
TraceElephant	Full-trace capture	**No attribution method at all**	We build the method	Full trace plus diagnosis
ErrorProbe	Best published: 59.5%	**Admits it fails on silent failures**; 3 backbones only	We adopt it, then extend	Unconditional check, verified repair, unseen architectures
AgentAsk	Prevent errors at handoffs	Blind to errors inside an agent	Reactive by design	Our own test of its idea failed (0.25) and is reported
PROTEA	Interactive debugging tool	Human approves every fix; **DAG-only**	Automated verification	Measured recovery rate; cyclic graphs
LangSmith / Langfuse / Braintrust / Arize / Maxim	Trace capture and evals	Human-driven diagnosis; no fix verification	We are the diagnosis layer above them	Attribution and verification they do not do
S24-011-D-HDLM (FYP)	Hallucination detection in one LLM's text	Single model; detection only	Multi-agent, attribution, repair	4 stages vs 1

4. Research Gap and Novelty

4.1 The rule we hold ourselves to

"No paper does X" is not, by itself, a reason to do X. Plenty of things are absent from the literature because they are pointless.

So for each claim we answer three questions: **why does it matter**, **how will we build it**, and **how will we measure whether it worked**. A claim that cannot answer all three is not a contribution.

4.2 The gap, broken into four parts

Gap 1 — Silent failures are structurally invisible

Evidence: ErrorProbe's own Limitations section. Its whole pipeline is anomaly-gated, so an error producing no anomaly is invisible **by construction**, not by accident. Every method in the lineage inherits the same gating assumption.

Why it matters: this is the failure mode with real consequences. A crash gets noticed. A confident wrong answer gets *acted on*.

How we build it: the constraint check runs **unconditionally and in parallel** with the tracer, never behind it. It recomputes the answer's checkable constraints from the task specification and compares.

How we measure it: precision and recall on injected silent failures, **plus abstention rate** (how often no checkable constraint exists), **plus false-positive rate on 15-20% deliberately clean controls**.

The boundary, stated before a panel states it for us: we do **not** claim to detect arbitrary semantic wrongness. We claim that *where the task specification yields machine-checkable constraints* — as it does for GSM8K's numeric answers and tau-bench retail's policy rules — a deterministic verifier catches well-formed-but-wrong outputs. Where no constraint exists, the module **abstains rather than guessing**, and the abstention rate is reported. That is what makes this neither trivial nor impossible.

Gap 2 — No paper tests an unseen architecture

Evidence: confirmed after full-text reading of all seven. ErrorProbe's "cross-domain transfer" claim is real but narrower than it looks — it is transfer across **task domains** (coding to maths) *within its own already-tested benchmark ecosystem*.

Why it matters: if a method only works on architectures it was tuned for, it is not a method, it is a fitted result.

How we build it: commit to CrewAI and native LangGraph — used by none of the seven papers. This is why the framework choice is a research decision, not a convenience one.

How we measure it: run the same pipeline on both, report the accuracy difference.

Gap 3 — Nobody verifies a fix before trusting it

Evidence: PROTEA proposes fixes but a human approves each one and it reports no recovery rate. No paper and no commercial platform measures whether a proposed repair actually works before it is applied.

Why it matters: an unverified fix can silently make things worse.

How we build it: re-execute downstream under a controlled replay protocol — frozen upstream context, pinned decoding parameters — and report a measured recovery rate over N replays.

How we measure it: recovery rate with variance. **CONFIRMED** — Spike 5 validated the mechanism: corrupted 97.2 recovered to 108.0.

Gap 4 — Nobody measures the false-positive rate

Evidence: absent from every paper and all seven commercial platforms.

Why it matters: without it, **a detector that flags everything scores well.** This is the check that stops "flag everything" looking like success.

How we build it: 15-20% deliberately clean controls in our own benchmark.

How we measure it: false-positive rate on those controls, provisional target under 5%.

4.3 Research vs engineering contributions

The handbook is explicit that novelty is **not** compulsory. Being clear about which is which is stronger than blurring them.

Contribution	Type	Why
Cross-architecture benchmark (500-1,000 traces, released)	**Research — data/resource**	A reusable artefact that did not exist
Silent-failure detection with measured abstention	**Research — method at an identified gap**	Targets a documented blind spot with a measurable claim
Verified repair with measured recovery rate	**Research — method at an identified gap**	Nobody reports this number
Reproducing ErrorProbe faithfully	**Research — replication**	The handbook names faithful reproduction with transparent reporting as a legitimate contribution
The SDK, backend, dashboard, deployment	**Engineering — system/architecture**	The handbook notes this is usually "the largest and most defensible contribution" in Stream A
Memory on/off ablation (if Tier 2 gate fails)	**Research — evaluation**	A negative result properly reported is still a result

4.4 Why this is a year of work, not a weekend

The AI-resilience test: *could a strong student with AI assistance build this in a week?* **No.**

AI can accelerate SDK scaffolding, a first pass at the graph data model, tests and docs. It cannot shortcut:

- Instrumenting two real multi-agent frameworks end-to-end, including their differing message-passing and tool-call semantics
- Running fault injection at scale to produce a labelled benchmark — data engineering effort, not a generation task
- Reproducing ErrorProbe faithfully enough that a head-to-head comparison is defensible
- Designing constraint-verification logic where the failure mode is precisely "looks fine, isn't"
- Measuring and defending trade-offs under real deployment constraints

Effort budget: ~1,130 person-hours (3 members x ~15 h/week x ~28 weeks, minus a 10% exam haircut). FYP-1 ~490 h, FYP-2 ~640 h.

4.5 The answer to "you just combined existing techniques"

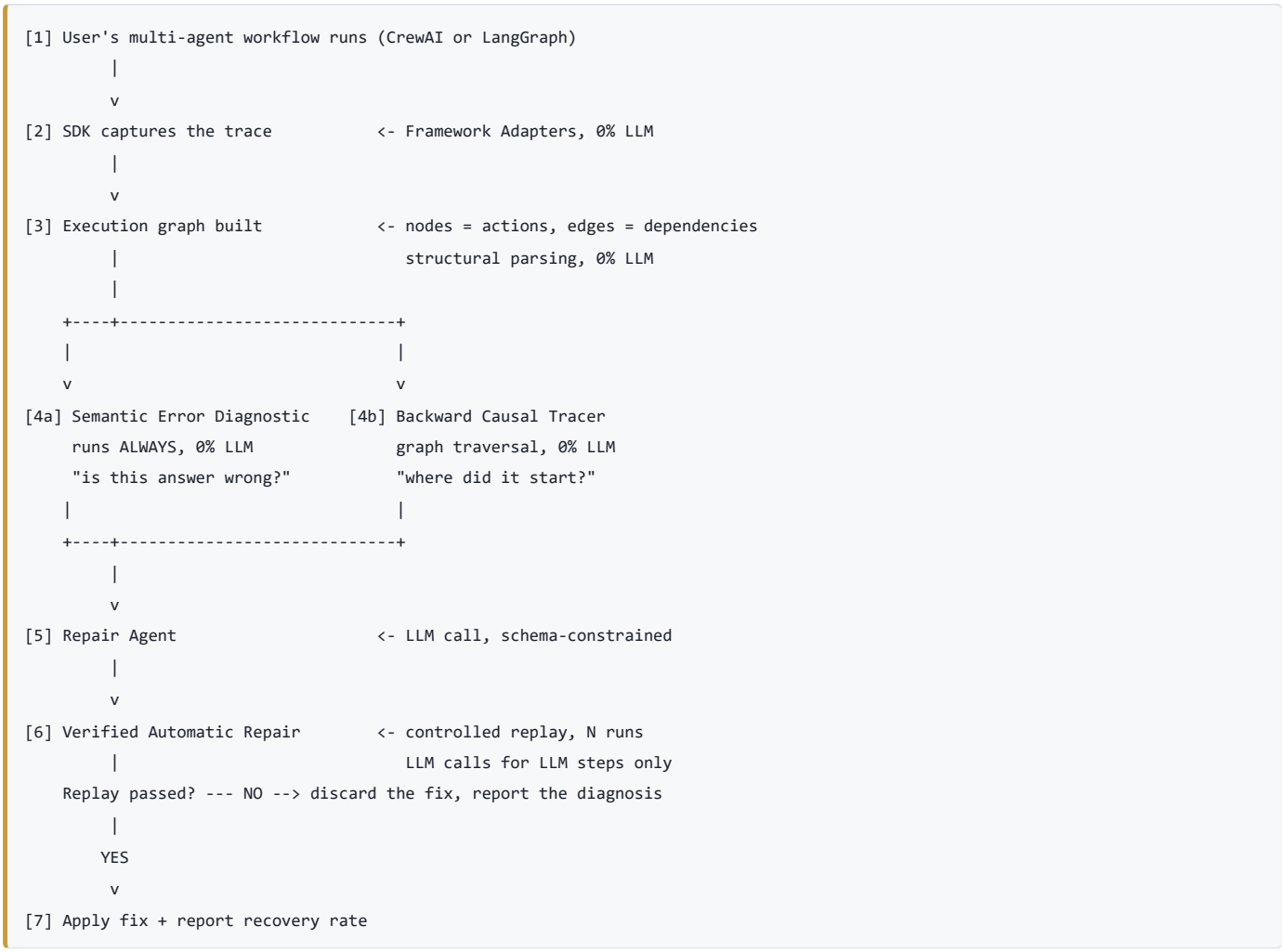
Say it plainly, first, before it is used against you:

> *"Correct — none of the individual mechanisms are new. Rule-based fault injection is AgenTracer's. Structural graph parsing is standard. Constraint checking is ordinary software engineering. The handbook itself says external components are expected, not exceptional. The question is never whether established techniques are used — it is what they are used to prove.*
>
> *Strip away every technique and ask what is left: a number. Either the pipeline holds up on an architecture it was never tuned for, or it does not. Either one deterministic check catches a failure class every anomaly-gated method admits it cannot see, or it does not. That number is the contribution — not the parts used to produce it."*

---

5. Implementation

5.1 The end-to-end flow



The load-bearing design fact: steps 2, 3, 4a and 4b make **zero LLM calls**. Only repair (5) and replay (6) do. Overall API dependence is targeted at ~15-20% of components, so the contribution survives swapping the LLM.

5.2 Feature-by-feature

F1 — Fault-Injection Harness

- **What happens:** pick a step in a working workflow, corrupt one value deterministically, record what was changed.
- **How:** rule-based operators. No LLM.
- **Components:** CrewAI/LangGraph hooks, an operator library.
- **Data:** GSM8K, tau-bench retail.
- **In / out:** clean workflow in; corrupted trace plus exact ground-truth label out.
- **Evaluation:** does the injected fault produce an observable, unambiguous failure?
- **Challenges: CONFIRMED problem** — Spike 3's first attempt corrupted a famous fact and the agent *self-corrected* it. Substring matching was too crude. Spike 3b fixed this using an unverifiable fabricated fact plus structured verification.
- **Fallback:** keep both behaviours as documented cases. Real fault behaviour includes self-correction, and the benchmark should reflect that rather than pretend otherwise.

F2 — Cross-Architecture Benchmark

- **What happens:** run injection across both frameworks at scale to produce labelled traces.

- **Components:** the harness plus a runner, plus a datasheet.
- **In / out:** task pools in; 500-1,000 labelled traces out, 15-20% clean controls.
- **Evaluation:** size, label correctness by construction, clean-control proportion.
- **Challenges:** volume and cost. **RESOLVED** — generation is only ~5% of the compute budget; reproduction *iteration* is ~63%.
- **Fallback:** pre-registered descope ladder drops 1,000 to 500 before anything else is cut. **300 cases with statistics beats 1,000 without.**

F3-F5 — Culprit Agent, Error Step Localization, Verification Agent

- **What happens:** the system hypothesises which agent and step failed, then checks the hypothesis against real tool-call evidence before committing.
- **Components:** ErrorProbe's Strategist and Investigator stages.
- **Data:** Who&When (184 cases) for reproduction.
- **Evaluation:** agent accuracy, step accuracy, propagation-path F1, against ErrorProbe's published numbers.
- **Challenges: CONFIRMED as the project's biggest risk.** Our simplified reproduction reached only 0-25% step accuracy against a published 59.5%.
- **Fallback: pre-registered acceptance band** — accepted within +/-10 points of 59.5% on a comparable-class backbone. Outside it, **both** numbers are reported side by side, declared in advance. The handbook names faithful reproduction with transparent reporting as a legitimate contribution. Either outcome is a result; only an unreported gap would be a failure.

F6 — Repair Agent

- **What happens:** generates a corrected value for the localised failure.
- **Components:** an LLM call constrained to emit one atomic field.
- **Evaluation:** repair success rate through actual downstream replay, not string matching.
- **Challenges: CONFIRMED** — Spike 8 got 4/6. Both failures returned a *derived downstream value* instead of the atomic field asked for (the discount amount instead of the discount rate; the order total instead of the quantity). **One cause, not two unrelated bugs.**
- **Fallback:** constrain the output schema to the atomic field. This is a concrete, scoped fix, already diagnosed.

F7 — Semantic Error Diagnostic (the headline)

- **What happens:** recompute the answer's constraints from the task spec, compare, flag or pass or abstain.
- **Components:** a rule-based checker. **Zero LLM calls inside the checker.**
- **Data:** GSM8K numeric answers; tau-bench retail policy rules and action sequences.
- **In / out:** trace plus task spec in; flag / pass / abstain out.
- **Evaluation:** precision and recall on injected silent failures; false-positive rate on clean controls; abstention rate as coverage.
- **Challenges: the honest one** — the live n=3 run produced **zero natural silent failures**, so real-world recall is unmeasured.
- **Fallback:** measure against *injected* near-misses at scale, where ground truth is exact by construction. That is what the fault-injection harness is for.

"If no paper implements this, how can we realistically implement and validate it?"

Answer in three beats:

1. **The mechanism is simple, not clever.** Recompute the answer from the task's own data and compare within a tolerance. That is why it needs zero LLM calls.
2. **It is already built and tested.** Spike 4b: 3 of 3 known-wrong values caught including a $20.00-vs-$19.75 near-miss, 0 of 3 correct values flagged including one differing only by legitimate rounding.
3. **The reason no paper does it is architectural, not technical.** They gate on an anomaly first. Removing the gate is a design decision, not an invention — which is exactly why it is achievable in a year.

F8 — Verified Automatic Repair

- **What happens:** re-run downstream with the candidate fix; apply only if the replay reproduces the expected result.
- **Components:** replay engine with frozen upstream context and pinned decoding parameters.
- **Evaluation:** recovery rate over N replays, with variance.
- **Challenges: LLM steps are not deterministic even at temperature 0.**
- **How we handle it honestly:** the **protocol** is deterministic — which steps re-run, with what context, under which parameters — and every non-LLM computation is bit-identical. LLM variance is **measured**, not assumed away. "Verified" means the recovery rate was measured. That is the correct response to a nondeterministic component, not a weaker claim.

F9 / F10 — Tier 2

Covered in Section 2.3. Both carry pre-registered gates. Present them as gated, never as done.

---

6. Dataset and Data Feasibility Audit

6.1 Status key

Marker	Meaning
CONFIRMED	Downloaded, counted, licence checked against a LICENSE file
TO BUILD	We generate it
BENCHED	Downloaded and verified, deliberately not used
BLOCKED	Cannot be used

6.2 The audit

Dataset	Purpose	Source	Available?	Usable?	Format	Labels / signals	Problems	Backup
Who&When	Baseline reproduction	ICML 2025 repo, arXiv:2505.00212	**CONFIRMED** — downloaded	**Yes** — MIT, LICENSE in tree	JSON traces plus annotations	Culprit agent, decisive step (human-annotated)	Two splits; downstream papers restate sizes inconsistently (Algo=127/Hand=58 vs HC=54/AG=126)	None needed — load-bearing and reliable
tau-bench (retail)	Injection substrate	sierra-research/tau-bench	**CONFIRMED**	**Yes** — MIT, verified	Task definitions plus full env code	Policy rules plus full correct action sequences	Richer, so more complex to instrument	GSM8K covers the numeric case
GSM8K	Injection substrate	openai/grade-school-math	**CONFIRMED** — 8,792 rows	**Yes** — MIT, verified 7 Sep 2026	Question plus single-integer answer	Exact numeric ground truth	Single-domain, arithmetic only	tau-bench covers the richer case
OUR benchmark	**The contribution**	Built by us	**TO BUILD**	Target 500-1,000 traces	Traces plus injection labels	Ground truth exact by construction	Volume is the main risk	Descope ladder: 1,000 to 500
TracerTraj-2.5K	Would have been baseline data	github.com/bingreeky/AgenTracer	**BLOCKED**	**No**	—	—	**NO LICENSE FILE EXISTS.** ErrorProbe described it as Apache 2.0; direct inspection found nothing. Only a 127-row sample is real	Who&When alone carries reproduction
MATH	Injection substrate	EleutherAI/hendrycks_math	CONFIRMED — 12,500	Yes, MIT	Equations, sets, intervals	Symbolic answers	**BENCHED** — answers fit the numeric near-miss check worse than plain integers	Not needed
MBPP+	Injection substrate	evalplus/mbppplus	CONFIRMED — 378	Permissive, confirm before release	Code tasks	Test-based	**BENCHED** — left over from the abandoned self-generation attempt	Not needed
GAIA	Injection substrate	gaia-benchmark/GAIA	CONFIRMED — 466	**RISKY**	—	—	**GATED. Redistribution NOT permitted.** May use, may not redistribute	Not needed

Dataset	Purpose	Source	Available?	Usable?	Format	Labels / signals	Problems	Backup
TraceElephant	Optional secondary	TraceElephant repo	Likely available	CC BY 4.0 for traces; code licence unspecified	Full input+output traces	Human-annotated	Not in the committed plan	Optional only

6.3 Data for silent-failure detection specifically

This is the question a panel is most likely to press, so handle it directly.

What the detector needs: a task whose specification yields a **machine-checkable constraint**, plus the correct answer computable independently of the model.

Substrate	Constraint available	Why it works
GSM8K	A single integer, recomputable from the question	The cleanest possible check: recompute, compare within tolerance
tau-bench retail	Policy rules plus a full correct action sequence	Richer — checks *behaviour*, not just a final number

The honest limitation: where the task yields **no** checkable constraint, the module **abstains**. It does not guess. Abstention rate is reported as a coverage metric, which is what makes the boundary of the claim measured rather than asserted.

Where the labelled silent-failure data comes from: we **inject** it. Natural silent failures are rare — our n=3 live run produced zero. The fault-injection harness produces near-misses on demand, with ground truth exact by construction. This is not a workaround; it is the same technique AgenTracer used to remove its annotation cost.

6.4 Data risks, stated plainly

Risk	Severity	Status
TracerTraj unavailable, so only one reproduction dataset	**High**	**CONFIRMED and accepted.** Reproduction rests on Who&When alone
GAIA is gated and cannot be redistributed	**Medium**	Must be **purged from git history** before this repo is ever public
Own benchmark may not reach 500	Medium	Descope ladder pre-registered
Natural silent failures are rare	**Confirmed**	Injection is the answer; recall in the wild stays unmeasured

7. CCP Mapping

> **CCP — Complex Computing Problem.** The Seoul Accord defines nine characteristics. The handbook requires "some or all," and its own worked "Strong" examples clear 6 to 9.

Granite self-rates 8 clear, 1 weak.

7.1 The nine characteristics

#	Characteristic	Status	Why	Evidence artefact
1	**Conflicting requirements**	**Clear**	Diagnostic depth vs latency; trace completeness vs privacy; generalisation vs specialisation	NFR: latency overhead target under 50%; privacy threat model
2	**No obvious solution**	**Clear**	Best published step accuracy is 59.5% after 15 months and 7 papers	Literature workbook, Sheet 3
3	**Depth of knowledge**	**Clear**	Needs execution-graph modelling, causal reasoning, distributed tracing, LLM evaluation methodology	Architecture sketch; Spikes 1, 2, 6
4	**Unfamiliar issues**	**Clear**	Delayed error manifestation, LLM non-determinism, cross-architecture transfer — not routine dev issues	R25 (determinism); Spike 6 (cycles)
5	**Beyond standard practice**	**Clear**	No off-the-shelf framework does attribution plus silent-failure detection plus verified recovery	Commercial comparison, 7 platforms
6	**Diverse stakeholders**	**WEAK**	Operators, downstream product teams, auditors are named — but **no real stakeholder has been contacted yet**	Scheduled for the Mid-2 acceptance study. **This is the one open gap between 8/9 and 9/9**
7	**Significant consequences**	**Clear**	A missed silent failure ships an incorrect answer that a person or system acts on	The Riverside Hardware case; the refund scenario
8	**Interdependence**	**Clear**	The entire problem is failures propagating through interdependent agents	The execution graph exists to model exactly this; Spike 2
9	**Ill-defined cause**	**Clear**	Three unknowns, named and resolved by distinct pipeline stages	See 7.2

7.2 Characteristic 9 — the paragraph to use verbatim

> At diagnosis time three things are unknown and must be investigated, not assumed: **(1) whether the trace contains a failure at all** — silent failures produce no error signal, so their presence is itself uncertain; **(2) which agent and step is the true origin**, which may be many steps upstream of where the failure becomes visible; and **(3) what the correct behaviour would have been**, which is recorded nowhere in the trace and must be hypothesised and tested. The three-stage pipeline — constraint-based flagging, then backward causal tracing, then controlled counterfactual replay — is the mechanism by which that uncertainty is progressively resolved, and the POC is the first evidence the resolution process works.

Each unknown maps to one stage. That mapping is the answer, not the list.

7.3 Feature-to-CCP map

Feature	CCP satisfied	Evidence
Fault-Injection Harness	3, 8	Spike 3/3b — unambiguous ground truth, propagation verified
Cross-Architecture Benchmark	3, 5, 7	Spike 2 — real CrewAI graph captured
Culprit Agent / Step Localization	2, 3, 9	Spike 1 — 5 real runs, reported as a range
Verification Agent	2, 3	Part of the ErrorProbe reproduction
Repair Agent	1, 5	Spike 8 — 4/6, both failures share one cause
Semantic Error Diagnostic	**1, 2, 5, 7, 9**	Spike 4 (live, 0 false positives) + Spike 4b (seeded, 3/3 caught)
Verified Automatic Repair	**1, 4, 9**	Spike 5 — 97.2 recovered to 108.0

Feature	CCP satisfied	Evidence
Recovery Controller	1, 8	**No POC.** Gated
Verified Repair Memory	3, 4	**One test, failed.** Gated
Cyclic graph handling	4, 8	Spike 6 — bounded terminates, unbounded hits the 10,000-step cap

7.4 The action still open

Characteristic 6 is weak and you should say so if asked. The fix is naming real stakeholders actually contacted. It is scheduled for the Mid-2 acceptance study — declared open, not claimed as done.

If asked: *"It is real but under-argued. We have literature evidence and a seven-vendor commercial market, but we have not put this in front of a working engineer yet, because we do not have a working system to show them. That is scheduled for Mid-2. We would rather declare it open than overclaim contact that has not happened."*

---

8. Panel Defense Q&A

Format: Question, then a short answer to memorise, then what to say if pushed, then the evidence.

8.1 Silent failures

Q1. What is a silent failure? Give an example.

- **Short:** A confident, well-formed, complete — and wrong — answer, with no error, no retry, no warning.
- **If challenged:** Our own captured run. Three agents were asked when Riverside Hardware closes. The answer said 6:15 PM; the truth was 9:40 PM. The Writer restated the wrong value and invented a justification for it. Nothing crashed.
- **Evidence:** `poc_spike3b_output.json` — propagation verdict `propagated_blindly`.

Q2. Why can't existing methods catch these?

- **Short:** They all start by looking for something visibly wrong, then reason backwards. If nothing looks wrong, the search never starts.
- **If challenged:** This is ErrorProbe's own Limitations section, not our characterisation. It is structural: the pipeline is anomaly-gated.
- **Evidence:** ErrorProbe (ACL 2026 Findings), Limitations.

Q3. How do you detect one, then?

- **Short:** The constraint check runs on **every** trace, not only ones that look broken. It recomputes the answer from the task's own data and compares.
- **If challenged:** That is the whole design change — remove the gate. Zero LLM calls in the checker.
- **Evidence:** Spike 4b — 3 of 3 known-wrong caught, 0 of 3 correct flagged.

Q4. Can you detect any semantically wrong answer?

- **Short: No, and we do not claim to.** Only where the task yields a machine-checkable constraint.
- **If challenged:** Where no constraint exists the module **abstains** rather than guessing, and we report the abstention rate as a coverage metric. That is what makes the boundary measured instead of asserted.
- **Evidence:** Plan section 7.0 item 7 — the scope of the claim, written before a panel asked.

Q5. Your spike caught nothing. Isn't that a failure?

- **Short:** Two different runs, and they must not be conflated. The **live** n=3 run had zero natural failures because the model got all three right — so recall in the wild is **unmeasured**. The **seeded** test caught 3 of 3.
- **If challenged:** That is exactly why the fault-injection harness exists. Natural silent failures are rare; we inject them so recall can be measured at a scale that means something.
- **Evidence:** `poc_spike4_silent_failure_output.json` and `poc_spike4b_mechanism_output.json`.

8.2 Detection vs correction

Q6. Do you detect, or fix?

- **Short:** Both — but the fix is verified before it is applied, and a human approves it.
- **If challenged:** Repair Agent generates a candidate; Verified Automatic Repair replays downstream; if the replay fails, the fix is discarded.
- **Evidence:** Spike 8 (generation, 4/6) chained into Spike 5 (replay).

Q7. What if the repair makes things worse?

- **Short:** It cannot be applied without passing the replay first. That is the entire point of verifying.
- **If challenged:** And every repair is human-gated — advisory only, never silently auto-applied.
- **Evidence:** FR4; the ethical statement on the deck.

8.3 Feasibility

Q8. If no paper does this, how can you?

- **Short:** Because the mechanism is simple, not clever. Recompute from the task's own data and compare. The reason no paper does it is architectural — they gate on an anomaly first.
- **If challenged:** Removing a gate is a design decision, not an invention. That is precisely why it is achievable in a year.
- **Evidence:** Spike 4b, already built and run.

Q9. Is this a year of work, or a weekend with AI help?

- **Short:** Instrumenting two frameworks end to end, generating a labelled benchmark at scale, and reproducing ErrorProbe faithfully are all data-engineering and evaluation-design effort. AI does not shortcut those.
- **If challenged:** ~1,130 person-hours budgeted against ~1,130 available. FYP-1 490, FYP-2 640.
- **Evidence:** Effort budget, plan section 7.2.

8.4 Data

Q10. Where does your data come from?

- **Short:** Who&When (184 cases, MIT, downloaded) for reproduction. GSM8K and tau-bench retail as injection substrates. Our own benchmark is the contribution.
- **If challenged:** Every licence checked against a LICENSE file, not a citation.
- **Evidence:** Literature workbook, Sheet 4.

Q11. Isn't TracerTraj a baseline for you?

- **Short: It was, until we checked. It was never released.** No LICENSE file, only a 127-row sample.
- **If challenged:** ErrorProbe's paper described it as Apache 2.0. We found nothing by direct inspection. That is our standing lesson: a citation is not a licence check.
- **Evidence:** R11.

Q12. Aren't you shaping the benchmark to flatter your detector?

- **Short:** A fair question, and partly yes by construction — so here is what limits it.
- **If challenged:** Three things. tau-bench was chosen for richer ground truth and domain match, not only numeric fit. Extension #1 adds two structurally different fault types the detector was not chosen to handle. And MATH was **benched specifically because** its answers fit our check *worse* — evidence we are tracking this tension, not blind to it.
- **Evidence:** datasets.md; plan section 8.

8.5 Evaluation

Q13. What do you measure?

- **Short:** Agent and step accuracy, propagation-path F1, silent-failure precision and recall, abstention rate, false-positive rate on clean controls, recovery rate, latency overhead.
- **If challenged:** The false-positive rate is absent from every paper and every platform we checked. Without it, a detector that flags everything scores well.
- **Evidence:** Plan section 11.

Q14. What counts as success?

- **Short:** Pre-registered. Reproduction accepted within +/-10 points of 59.5% on a comparable-class backbone.
- **If challenged:** Pre-registered means written down before any result existed. Outside the band, **both** numbers are reported side by side. Either outcome is a result; only an unreported gap would be a failure.
- **Evidence:** Plan section 11, dated before the run.

Q15. Your step accuracy was 0-25%. ErrorProbe gets 59.5%. Explain.

- **Short:** That is a simplified **two-stage** reproduction of a **three-stage** pipeline, on a local 3B model, on 8 cases. It is not a like-for-like comparison and we do not present it as one.
- **If challenged:** We report it as a range across 5 runs, not one lucky number, because variance at n=8 is real. Closing that gap is the top FYP-1 priority.
- **Evidence:** `poc_spike1_multirun_summary.json`.

Q16. False positives and false negatives?

- **Short:** False positives measured on 15-20% deliberately clean controls, provisional target under 5%. False negatives — recall — measured on injected silent failures.
- **If challenged:** In the wild, recall is currently **unmeasured**. We say so rather than implying otherwise.
- **Evidence:** Spike 4 summary.

8.6 Novelty and difference

Q17. What is actually new?

- **Short:** Not the attribution algorithm — we adopt ErrorProbe's. Three things: the unconditional silent-failure check with reported abstention, repair verified by measured replay, and cross-architecture testing on CrewAI and LangGraph.
- **If challenged:** The gap is the **combination**, evaluated as one pipeline. No system confirms all four properties together.
- **Evidence:** Literature workbook, Sheet 5.

Q18. Isn't this just gluing existing techniques together?

- **Short:** Yes, and the handbook says external components are expected. The gluing is not the claim — the measurement is.
- **If challenged:** Strip away every technique and what is left is a number. Either it generalises past its training ecosystem or it does not.
- **Evidence:** Plan section 4.1.

Q19. A prior FYP already did hallucination detection.

- **Short:** S24-011-D-HDLM, Spring 2024. We found it ourselves and it confirms the problem is real.
- **If challenged:** It covers one of our four stages, on one model's text. It has no notion of which agent, which step, or whether a fix works — those concepts do not exist without multiple interacting agents.
- **Evidence:** 660-title archive check.

Q20. LangSmith and Arize are shipping automated diagnosis. Aren't you too late?

- **Short:** They sharpen the differentiation rather than removing it. Neither publicly documents counterfactual fix-testing or general silent-failure detection.
- **If challenged:** Arize Alyx is marketing-stage with an undocumented mechanism. We position as a diagnosis layer complementary to these platforms, not a competing observability product.
- **Evidence:** Commercial comparison, 7 platforms.

8.7 Architecture and agents

Q21. Why CrewAI and LangGraph?

- **Short:** Because **no reviewed paper used either.** That makes it a real test of generalisation rather than a convenience choice.
- **If challenged:** It is a research decision. Picking a framework a paper already used would make the transfer claim meaningless.
- **Evidence:** Literature workbook, Sheet 5, Gap 2.

Q22. How much of this is just calling an LLM?

- **Short:** Target ~15-20% of components. Trace capture, graph building, the constraint check and backward tracing all make **zero** LLM calls.
- **If challenged:** That is why the contribution survives swapping the model. Only repair generation and replay use one.
- **Evidence:** Plan section 6, per-component classification.

Q23. What if the agents loop?

- **Short:** Handled and tested. Bounded traversal terminates on a real cycle.
- **If challenged:** Spike 6 ran both: bounded terminated in 12 steps with all 5 nodes reached; the unbounded control never terminated and hit the 10,000-step cap. The control is what proves the bound is doing real work. PROTEA is explicitly DAG-only.
- **Evidence:** `poc_spike6_output.json`.

8.8 Limitations and failure cases

Q24. What is actually implemented versus simulated?

- **Short:** Eight spikes ran with real committed output files. The end-to-end pipeline is **not** wired together yet — that is FYP-1 implementation work.
- **If challenged:** Every number we present traces to a JSON file you can open. Nothing is simulated. What is not built, we say is not built.
- **Evidence:** `dev/spikes/poc/` — 9 scripts and outputs.

Q25. What if the whole thing does not work?

- **Short:** Each risky item has a pre-registered fallback written before the result existed.
- **If challenged:** Reproduction misses the band, both numbers reported. Recovery Controller fails its gate, resumption ships via controlled replay, which Tier 1 already delivers. Repair Memory fails, it ships as a memory-on-vs-off ablation — an evaluation contribution.
- **Evidence:** Tier 2 gate table; descope ladder.

Q26. Which part are you least confident about?

- **Short:** Two. Reproducing 59.5%, and the Recovery Controller, which has no POC at all.
- **If challenged:** Verified Repair Memory has one test and it **failed** — 0 of 8 committed, accuracy regressed 75% to 50%. It is in the core with a gate, not smoothed over.
- **Evidence:** R18, R21.
- **Note:** answering this honestly is worth more than a confident dodge. The panel is testing whether you know your own weak points.

Q27. Your project runs on a local 3B model. Is that a real evaluation?

- **Short:** No, and it is not meant to be. The spikes were feasibility checks at zero cost.
- **If challenged:** The reproduction that matters is budgeted for a comparable-class backbone — the fair comparator to ErrorProbe's published setup. Running spikes locally is why all eight cost nothing.
- **Evidence:** cost-estimate.md; R37.

Q28. You report a negative result. Why keep it?

- **Short:** Spike 7's preventive classifier scored 0.25 — near-random. We lead with it.
- **If challenged:** The handbook's own guidance says a POC that disproves an assumption is more valuable than one that confirms it. It is also why the reactive path is the committed one and the preventive classifier is only an extension.
- **Evidence:** `poc_spike7_output.json`.

8.9 CCP

Q29. Which CCP characteristics do you claim?

- **Short:** Eight clear, one weak. The weak one is diverse stakeholders.
- **If challenged:** We have literature and a seven-vendor market but no engineer has used it yet, because there is nothing working to show them. Scheduled for Mid-2. Declared open rather than overclaimed.
- **Evidence:** Plan section 2.

Q30. Justify "ill-defined cause."

- **Short:** Three unknowns: whether it failed at all, which step, and what the correct answer was. Each is resolved by a distinct pipeline stage.
- **If challenged:** Use the verbatim paragraph in Section 7.2.

---

9. The Final Product

9.1 What it is

An installable SDK plus a diagnosis dashboard, deployed in Docker, operable by someone who did not build it.

9.2 Main workflow

```
INSTALL      pip install granite; point at your CrewAI/LangGraph workflow
   |
RUN          your workflow runs normally; the SDK captures the trace
   |
DIAGNOSE     did it fail? which agent? which step? what is the evidence?
   |
REPAIR       a candidate fix is generated and replayed downstream
   |
REVIEW       operator sees the diagnosis and the replay result, approves or rejects
   |
RESUME       verified repair applied; execution resumes from a safe point
```

9.3 Inputs and outputs

Inputs	A running CrewAI or LangGraph workflow; the task specification; optional constraint rules
Outputs	A diagnosis record — culprit agent, failing step, confidence, propagation path, evidence — plus a verified repair with a measured recovery rate

9.4 What the panel sees in the demo

1. **The failing run.** A three-agent workflow produces a confident wrong answer. Nothing crashes. Let that sit for a second.
2. **The execution graph.** Nodes are agent actions, edges are dependencies, built from the real trace.
3. **The constraint check fires.** With no anomaly to trigger it — that is the point.
4. **Backward tracing.** The tracer walks the edges in reverse and lands on the origin node, not the node where the symptom appeared.
5. **The repair, replayed.** Corrupted 97.2, recovered 108.0, `recovery_success = true`.
6. **The evidence page.** All eight spikes with their real output files, including the negative one.

Say during the demo: which parts are live and which are replayed from committed spike outputs. Do not let a panel discover that distinction on its own.

9.5 Screens

Screen	Shows
Run list	Traces captured, pass/fail/flagged
Diagnosis detail	Execution graph, culprit, evidence, confidence
Repair review	Proposed fix, replay result, approve/reject

10. Reality Check

This section exists to be read before a panel writes it for you.

10.1 Unsupported or inconsistent claims — fix before the defense

Issue	Reality	Status
"9+ papers read in full"	**Seven** were read in full; three more (GraphTracer, FALAT, DoVer) were triaged at abstract level only. The error was attaching "read in full" to all ten	**FIXED** across 13 files including the deck. Say "seven read end-to-end, three triaged"
"Catches a near-miss" on the spike table	True, but it came from the **seeded** mechanism test, not the live n=3 run, which caught nothing because nothing was wrong	**FIXED** — the row now says "seeded test." Keep both halves in the spoken answer
Latency target under 50%	A target, not a measurement. Nothing has been measured yet	**OPEN.** Present as a target. Never imply it is achieved
CCP characteristic 6 (diverse stakeholders)	Rated **weak** by the project itself. No real stakeholder has been contacted	**OPEN, and correctly declared.** Scheduled for the Mid-2 acceptance study. See Section 7.4 for the wording

10.2 Weak novelty claims

Claim	How weak	Honest framing
"Cross-architecture generalisation is novel"	It is novel **as a test**, not as a technique. Running on a new framework is engineering	The *measurement* is the contribution — nobody has produced this number
"Verified Repair Memory is different from ErrorProbe's"	Defensible, but a panel will probe it	Different content, position and purpose. Rehearse this
"Cyclic graph support"	An extension, not core, though a POC exists	Do not present as a deliverable

10.3 Missing or risky data

Item	Status
TracerTraj-2.5K	**BLOCKED.** Never released. Reproduction rests on Who&When alone
GAIA	**GATED.** Must be purged from git history before the repo goes public
Own benchmark	Does not exist yet. The largest single build risk
Natural silent-failure data	Rare. n=3 produced zero. Injection is the only path

10.4 Features that should be simplified or watched

Feature	Problem	Recommendation
Recovery Controller	**No POC at all**	Keep the gate. If it slips, take the fallback early rather than late
Verified Repair Memory	**One test, failed.** Accuracy regressed 75% to 50%	Be ready to present the ablation as the outcome
13 optional extensions	Far too many to build	Present as a prioritised backlog, never as promises
Preventive classifier	Already failed at 0.25	Keep it as a reported negative. Do not revive it as a feature

10.5 Implementation risks

Risk	Severity	Mitigation
ErrorProbe reproduction misses 59.5%	**High**	Pre-registered +/-10 band; dual reporting
Benchmark stalls below 500	Medium	Hard floor 500; 300 with statistics beats 1,000 without
Backbone confound across papers	Medium	Controlled inside our own experiments; both numbers reported

Risk	Severity	Mitigation
A new paper closes the gap first	Medium	Re-check at each of the 5 gates; report honestly. The benchmark and SDK remain load-bearing either way
Local-model-only budget	Medium	Spikes cost $0. Reproduction needs a comparable-class backbone — confirm access early

10.6 Missing evaluation methods

Gap	Status
Human baseline	Planned, not run
Latency overhead	Never measured. Spike 2's timestamps were captured post-hoc, so they are **not** trustworthy for latency
Recall on natural silent failures	**Unmeasured, and likely to stay that way.** Injected recall is the measurable version
Real stakeholder validation	Scheduled Mid-2. This is CCP characteristic 6's weakness

10.7 The three things most likely to be attacked

1. **"Your step accuracy is 13%, theirs is 59.5%."** Simplified 2-stage vs 3-stage, 3B local model, n=8, reported as a range. Closing it is FYP-1's top priority.
2. **"You never caught a real silent failure."** Correct. Zero natural failures occurred in n=3. The seeded test measures the mechanism; injection at scale is how recall gets measured.
3. **"Two of your ten core features have almost no evidence."** Correct, and both carry gates written before results existed, with fallbacks that still close the loop.

In all three cases the answer is the same shape: concede the fact, then show it was anticipated and planned for. That is a stronger position than a confident denial, and it is the honest one.

Appendix — evidence index

Claim	File
Agent 62.5-85.7%, step 0-25%, 5 runs	`dev/spikes/poc/poc_spike1_multirun_summary.json`
Real CrewAI graph, 3 nodes 2 edges	`poc_spike2_trace_output.json`
Fault propagated blindly, ground truth recoverable	`poc_spike3b_output.json`
Live silent-failure run, 0 natural failures, 0 false positives	`poc_spike4_silent_failure_output.json`
Seeded mechanism test, 3/3 caught, near-miss included	`poc_spike4b_mechanism_output.json`
Corrupted 97.2 recovered to 108.0	`poc_spike5_output.json`
Bounded traversal terminates; unbounded hits 10,000-step cap	`poc_spike6_output.json`
Preventive classifier 0.25 — honest negative	`poc_spike7_output.json`
Repair agent 4/6, one shared failure cause	`poc_spike8_output.json`
7 papers, numbers, datasets, gap	`Literature_Review_Sheet.xlsx`
Commercial platforms and prior FYPs	`Commercial_Product_Comparison.md`
Risks, assumptions, decisions	`risk-register.md`